



Debido a los errores, decidí cambiar la pantalla a una lcd 16 x 02 y con controlador SPLC780D1 o compatible con HD44780. ¿Qué cambios debo realizar en el código para adaptarme a esta pantalla?

Pasar a una pantalla LCD 16x2 con controlador HD44780 es un cambio tecnológico importante. Dejas de trabajar con una matriz de píxeles (donde podías hacer letras gigantes de 60px) para trabajar con una cuadrícula fija de 16 columnas y 2 filas.

Además, el método de desplazamiento cambia por completo. El controlador HD44780 tiene un límite interno de 40 caracteres por línea; como tu poema es mucho más largo, no podemos usar la función de desplazamiento por hardware (hardware scroll). Debemos crear un "desplazamiento por software" recortando el texto a medida que avanza.

Cambios de Conexión (Muy Importante)

A menos que tu LCD tenga soldado un módulo negro en la parte trasera (módulo I2C), esta pantalla no usa solo 4 cables. Requiere cableado paralelo:

VSS: GND

VDD: 5V

V0 (Contraste): Pin central de un potenciómetro de 10k. Los extremos del potenciómetro van a 5V y GND. **Si no conectas esto, la pantalla se verá en blanco o con cuadrados negros fijos.**

RS: Pin 12 del Arduino

RW: GND (solo vamos a escribir, no a leer)

E (Enable): Pin 11

D4 a D7: Pines 5, 4, 3, 2

A (Ánodo LED): 5V (puedes poner una resistencia de 220 ohmios)

K (Cátodo LED): GND

Código para LCD 16x2

Este código hace que el texto se deslice como una marquesina, entrando por la derecha de la primera fila, saltando a la segunda fila de forma continua.

C++



```
#include <LiquidCrystal.h>
```

```
// Inicializa la librería con los pines: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
// El texto sin tildes ni eñes (los LCD estándar muestran símbolos japoneses)
// Hemos añadido 32 espacios al final para que la pantalla se limpie suavemente
String poema = "Luces bajas y escaleras de lava negra me toman la mano y es
```

```
int posicion = 0;
int longitudPoema;
```

```
void setup() {
  lcd.begin(16, 2); // Configura las 16 columnas y 2 filas
  longitudPoema = poema.length();
}
```

```
void loop() {
  lcd.clear();
```

```
  // Imprime 16 caracteres en la primera línea
  lcd.setCursor(0, 0);
  lcd.print(poema.substring(posicion, posicion + 16));
```

```
  // Imprime los 16 caracteres siguientes en la segunda línea
  // Esto da el efecto de que el texto fluye de arriba hacia abajo
  lcd.setCursor(0, 1);
  lcd.print(poema.substring(posicion + 16, posicion + 32));
```

```
  // Avanza un carácter
  posicion++;
```

```
  // Si ya mostramos todo (incluyendo los espacios finales), reiniciamos
  if (posicion >= longitudPoema - 32) {
    posicion = 0;
  }
```

```
  // Velocidad de lectura. Un LCD no actualiza tan rápido como un OLED,
  // 350 milisegundos por letra permite leer sin marearse.
```

```
delay(350);  
}
```

Nota si usas Módulo I2C: Si tu pantalla 16x2 sí trae el adaptador I2C integrado en la parte trasera, no uses este código ni el cableado complejo. Avísame y lo adaptaremos para usar la librería `LiquidCrystal_I2C` (que volverá a usar solo 4 cables al igual que tu OLED anterior).

cabe mencionar que estoy ocupando en esta oportunidad un Arduino 101

El código y la librería `LiquidCrystal` funcionarán exactamente igual sin necesidad de modificar una sola línea, ya que dependen de funciones estándar (`digitalWrite`) que el procesador Intel Curie del Arduino 101 maneja sin problemas usando la misma numeración de pines.

El único factor crítico al hacer este cambio físico reside en los niveles lógicos de voltaje. De la misma manera en que los umbrales de voltaje alto y bajo son vitales al acoplar integrados CMOS como un 4017 o un 4093 en un circuito, el Arduino 101 presenta una arquitectura particular: sus pines de entrada y salida operan a 3.3V (aunque son tolerantes a recibir 5V).

Para que la pantalla funcione correctamente con el Arduino 101:

Alimentación (VDD y Ánodo): Deben ir conectados obligatoriamente al pin de 5V del Arduino. Si los conectas a 3.3V, el contraste no será suficiente para mostrar los caracteres, sin importar cómo ajustes el potenciómetro.

Señales de datos (RS, E, D4-D7): El Arduino enviará pulsos de 3.3V al LCD. El controlador HD44780 estándar reconoce cualquier señal por encima de 2.2V como un nivel lógico ALTO, por lo que interpretará los comandos correctamente sin necesidad de añadir conversores de nivel lógico en el protoboard.

Maneja las conexiones con los mismos pines (12, 11, 5, 4, 3, 2) del esquema anterior y el poema comenzará a desplazarse por las dos líneas.

¿El texto parpadea o muestra caracteres extraños?

Quiero que el texto se visualice en la línea superior de la pantalla. En la segunda línea solo debe verse una línea punteada que avance al mismo ritmo que el texto.

Para lograr este efecto visual, vamos a modificar la lógica de la segunda línea. En lugar de imprimir más texto, generaremos un patrón matemático que dibuje la línea punteada carácter por carácter.

Además, he aplicado una pequeña mejora técnica: he eliminado la función `lcd.clear()` y en su lugar sobrescribimos las líneas constantemente. En las pantallas LCD 16x2, usar `lcd.clear()` en cada ciclo suele causar un parpadeo constante y molesto a la vista. Al sobrescribir los 16 caracteres de golpe, el desplazamiento será mucho más fluido y sólido.

Aquí tienes el código adaptado para tu Arduino 101:

C++



```
#include <LiquidCrystal.h>
```

```
// Pines para el Arduino 101 (los mismos que veníamos usando)
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
// Añadimos 16 espacios al final para que el texto salga de la pantalla suav
String poema = "Luces bajas y escaleras de lava negra me toman la mano y es
```

```
int posicion = 0;
int longitudPoema;
```

```
void setup() {
  lcd.begin(16, 2);
  longitudPoema = poema.length();
}
```

```
void loop() {
  // --- FILA SUPERIOR: Desplazamiento del texto ---
  lcd.setCursor(0, 0);
```

```

lcd.print(poema.substring(posicion, posicion + 16));

// --- FILA INFERIOR: Línea punteada animada ---
lcd.setCursor(0, 1);
for (int i = 0; i < 16; i++) {
    // Usamos la operación módulo (%) para crear una secuencia par/impar.
    // Al sumarle la 'posicion' actual, el patrón avanza a la misma velocidad
    if ((posicion + i) % 2 == 0) {
        lcd.print('-'); // Puedes cambiar este '-' por un '.' si prefieres pur
    } else {
        lcd.print(' ');
    }
}

// Avanzamos un "paso"
posicion++;

// Reiniciamos cuando el texto (incluyendo los espacios finales) haya pasado
if (posicion >= longitudPoema - 16) {
    posicion = 0;
}

// Velocidad de la animación (350ms suele ser cómodo para leer)
delay(350);
}

```

¿Cómo funciona la animación de la línea punteada?

En lugar de crear un `String` gigante con guiones, usamos un bucle `for` de 16 pasos junto con la operación módulo (`% 2`). Esto verifica si la suma de la posición actual más el número de la columna en la que estamos parados es par o impar.

El resultado es que en el primer "cuadro" de la animación verás `- - - -`, pero en el siguiente ciclo, como la variable `posicion` aumentó en 1, el patrón se invierte y dibuja `- - - -`. Esto engaña al ojo creando la ilusión óptica de que la línea se está moviendo hacia la izquierda de forma sincronizada con el poema.

¿Quieres controlar la velocidad de desplazamiento con un potenciómetro?

↔ Sí

En la línea 28 "lcd.print('-'); // Puedes cambiar este '-' por un '.' si prefieres puntos"

Quiero poder introducir el código ascii en Arduino IDE y que despues se renderice, por ejemplo ascii 254

Para enviar un código ASCII numérico directamente y que la pantalla lo dibuje como un símbolo en lugar de imprimir los números, debes cambiar la función

`lcd.print()` por `lcd.write()` .

Si usas `lcd.print(254)` , el Arduino convertirá ese número en texto y escribirá literalmente "254" en la pantalla ocupando tres espacios. En cambio,

`lcd.write(254)` envía el valor crudo (byte) al controlador HD44780, indicándole que busque el símbolo almacenado en esa posición de su memoria.

En la mayoría de las pantallas con controlador HD44780, el código 254 corresponde a un bloque sólido negro (■).

Aquí tienes el código modificado:

C++



```
#include <LiquidCrystal.h>
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
String poema = "Luces bajas y escaleras de lava negra me toman la mano y es
```

```
int posicion = 0;  
int longitudPoema;
```

```
void setup() {  
  lcd.begin(16, 2);  
  longitudPoema = poema.length();  
}
```

```
void loop() {  
  lcd.setCursor(0, 0);  
  lcd.print(poema.substring(posicion, posicion + 16));  
  
  lcd.setCursor(0, 1);  
  for (int i = 0; i < 16; i++) {
```

```

if ((posicion + i) % 2 == 0) {
    // Usamos lcd.write() para enviar el código numérico del símbolo
    lcd.write(254);
} else {
    lcd.print(' ');
}
}

posicion++;

if (posicion >= longitudPoema - 16) {
    posicion = 0;
}

delay(350);
}

```

Nota sobre el mapa de caracteres: Los valores del 0 al 127 corresponden al estándar ASCII (letras y números normales). Los valores del 128 al 255 (donde está el 254) varían según el fabricante de la pantalla; algunas tienen caracteres japoneses (Katakana) y otras símbolos europeos o matemáticos. Puedes probar con números como el 165 (suele ser un punto centrado) o el 223 (símbolo de grado °) para ver qué arroja el mapa de tu pantalla específica.

¿Quieres dibujar tu propio símbolo personalizado pixel por pixel?

↪ Sí

Quiero agregar un Potenciómetro conectado al Arduino mediante el pin A0, de manera que pueda leer el voltaje resultante.

✓

Integrar un control físico para interactuar con la lectura le dará una dimensión táctil excelente a tu proyecto. Este tipo de interacción es ideal si planeas ensamblar la pantalla y el potenciómetro en una de tus cajas modulares o prototipos de control

con piezas impresas en 3D, ya que permite al usuario explorar el texto a su propio ritmo.

Para lograr esto, usaremos la función matemática `map()` de Arduino. Esta función toma el rango del potenciómetro y lo convierte proporcionalmente en los tiempos de espera (`delay`), de modo que al girar hacia los extremos, la velocidad aumente.

Nota importante de hardware para tu Arduino 101: Conecta los pines de los extremos del potenciómetro a **GND** y a **3.3V** (no a 5V). El pin central va a **A0**. Como el conversor analógico del Arduino 101 funciona a 3.3V lógicos, si usas 5V podrías saturar la lectura o dañar el pin.

Aquí tienes el código con la nueva lógica implementada:

C++



```
#include <LiquidCrystal.h>
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
String poema = "Luces bajas y escaleras de lava negra me toman la mano y es
```

```
int posicion = 0;
int longitudPoema;
```

```
void setup() {
  lcd.begin(16, 2);
  longitudPoema = poema.length();
}
```

```
void loop() {
  // Leemos el valor del potenciómetro (0 a 1023)
  int valorPot = analogRead(A0);
  int retardo;

  // --- RENDERIZADO VISUAL ---
  lcd.setCursor(0, 0);
  lcd.print(poema.substring(posicion, posicion + 16));

  lcd.setCursor(0, 1);
  for (int i = 0; i < 16; i++) {
    if ((posicion + i) % 2 == 0) {
      lcd.write(254); // Tu símbolo ASCII
    } else {
      lcd.print(' ');
    }
  }
}
```

```

}

// --- LÓGICA DE MOVIMIENTO Y VELOCIDAD ---

// Hemos creado una pequeña "zona muerta" entre 490 y 530.
// Las lecturas analógicas suelen oscilar un poco. Si usamos exactamente 5
// el texto vibraría adelante y atrás cuando sueltas la perilla en el cent

if (valorPot > 530) {
  // AVANZA (hacia adelante)
  posicion++;
  if (posicion > longitudPoema - 16) {
    posicion = 0; // Reinicia al principio
  }
  // Mapeamos el valor: de 531 (lento, 600ms) a 1023 (rápido, 50ms)
  retardo = map(valorPot, 531, 1023, 600, 50);

} else if (valorPot < 490) {
  // RETROCEDE (hacia atrás)
  posicion--;
  if (posicion < 0) {
    posicion = longitudPoema - 16; // Va al final del texto
  }
  // Mapeamos el valor: de 489 (lento, 600ms) hasta 0 (rápido, 50ms)
  retardo = map(valorPot, 489, 0, 600, 50);

} else {
  // ZONA CENTRAL: El texto se detiene para que el usuario pueda leer
  retardo = 100; // Breve pausa para no saturar el procesador
}

// Aplicamos la velocidad resultante
delay(retardo);
}

```

¿Qué está pasando en este código?

1. **Límites Inteligentes:** Cuando el texto retrocede y llega a la posición 0, la variable `posicion` se ajusta automáticamente al final del texto (`longitudPoema - 16`). Esto crea un bucle infinito perfecto en ambas direcciones.
2. **Zona de Descanso:** Añadí intencionalmente un margen en el centro (entre 490 y 530). En lugar de saltar instantáneamente de retroceso a avance, el potenciómetro tiene un punto central donde la animación se pausa, lo que permite fijar la lectura en una frase específica cómodamente.

3. **Mapeo Invertido:** En la función `map()` del retroceso (`map(valorPot, 489, 0, 600, 50)`), el rango disminuye hacia cero, pero la velocidad aumenta (el `delay` cae a 50ms). A medida que giras la perilla más hacia la izquierda, el bucle itera más rápido.

Que estructura debo agregar si quiero que se muestra algo en la pantalla. Considera que estoy probando desde 0

Para probar tu pantalla desde cero con un texto estático, necesitas la estructura fundamental de la biblioteca "LiquidCrystal". Esta estructura se divide en tres partes esenciales: la inclusión de la biblioteca, la configuración inicial en el `setup()` y la ejecución continua (que en este caso quedará vacía).

Carga este código básico. Mantengo la misma configuración de pines que discutimos anteriormente:

C++



```
// 1. Incluir la biblioteca que maneja la comunicación
#include <LiquidCrystal.h>

// 2. Indicarle al Arduino a qué pines físicos está conectada la pantalla
// El orden es: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  // 3. Inicializar la pantalla especificando su tamaño (16 columnas, 2 filas)
  lcd.begin(16, 2);

  // 4. Ubicar el cursor (columna 0, fila 0 es la esquina superior izquierda)
  lcd.setCursor(0, 0);
  lcd.print("Prueba de LCD"); // Imprimir en la primera línea

  // 5. Mover el cursor a la segunda línea (columna 0, fila 1)
  lcd.setCursor(0, 1);
  lcd.print("Sistema OK");    // Imprimir en la segunda línea
}

void loop() {
```

```
// Para una prueba básica donde el texto no se mueve,  
// el loop se deja completamente vacío.  
// Lo que escribiste en el setup() permanecerá fijo en la pantalla.  
}
```

Los 3 componentes críticos de esta estructura:

La Declaración Global (`LiquidCrystal lcd(...)`): Va antes del `setup()` . Es obligatoria porque crea el "objeto" llamado `lcd` y establece el puente físico entre tu Arduino y el controlador HD44780.

La Inicialización (`lcd.begin(16, 2)`): Va dentro del `setup()` . Enciende el controlador y ajusta el mapa de memoria interno para 16 columnas y 2 filas. Si omites esta línea, no se imprimirá nada.

Posicionamiento (`lcd.setCursor(x, y)`): Va antes de cualquier instrucción de impresión. A diferencia del Monitor Serie del ordenador, la pantalla LCD no hace saltos de línea automáticos. Si no le dices dónde empezar a escribir, sobrescribirá lo anterior o el texto desaparecerá fuera de los límites visibles.

ayudame a solucionar el código y optimizarlo

```
#include <LiquidCrystal.h>  
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

✓

El problema principal de tu código es que el `loop()` se ejecuta miles de veces por segundo sin ninguna pausa. El Arduino imprime los dos primeros textos, borra la pantalla una fracción de milisegundo después con `lcd.clear()` e imprime el tercer texto. Esto genera un parpadeo constante donde probablemente solo logres ver el último mensaje o la pantalla con un contraste extraño.

Además, por los nombres de tus variables (`textoInicial`), esta secuencia de créditos debería mostrarse solo una vez al arrancar, por lo que su lugar correcto es dentro del `setup()` .

Aquí tienes el código corregido estructuralmente:

C++



```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Usar 'const' ayuda a optimizar la memoria del microcontrolador
const char textoInicialA[] = "[PLACEHOLDER]-CC";
const char textoInicialB[] = "Cuando nuestras ";
const char textoInicialC[] = "E. BARRETT BROWN";

void setup() {
  // Serial.begin(9600); // Solo es necesario si vas a leer datos en el comp
  lcd.begin(16, 2);

  // --- PRIMERA PANTALLA ---
  lcd.setCursor(0, 0);
  lcd.print(textoInicialA);

  lcd.setCursor(0, 1);
  lcd.print(textoInicialB);

  // Pausa de 3 segundos para que el usuario alcance a leer
  delay(3000);

  // Limpiamos la pantalla
  lcd.clear();

  // --- SEGUNDA PANTALLA ---
  lcd.setCursor(0, 0);
  lcd.print(textoInicialC);

  // Pausa de 3 segundos
  delay(3000);

  // Limpiamos nuevamente, dejando la pantalla lista para el programa princi
  lcd.clear();
}

void loop() {
  // Aquí irá tu programa continuo (por ejemplo, el texto que se mueve con e
}
```

Problemas que debes considerar

Límite de 16 caracteres: Tu LCD tiene un límite estricto de 16 espacios por línea. Textos como *"Cuando nuestras dos almas se eleven"* tienen 35 caracteres. Si usas un `lcd.print()` estático, el controlador HD44780 simplemente cortará todo lo que pase de la letra 16 de forma invisible. En el código de arriba, recorté temporalmente tus frases para que calcen.

Gestión del borrado: La instrucción `lcd.clear()` toma bastante tiempo de procesamiento para el controlador de la pantalla en comparación con imprimir un carácter. Por eso, siempre debe estar acompañada de bloqueos de tiempo (`delay`) o condicionales, nunca escribirse "suelta" dentro de un `loop()` rápido.

¿Quieres que estos textos largos se desplacen?

↔ Sí

Quiero que `textoInicialA` ocupe `lcd.setCursor(0,0)` y `(0,1)`

El controlador HD44780 de tu pantalla no hace un "salto de línea" automático. Si intentas imprimir un texto de más de 16 letras seguidas en la fila 0, las letras sobrantes no bajarán a la fila 1, sino que se escribirán en una "memoria invisible" y desaparecerán de tu vista.

Para lograr que `textoInicialA` ocupe ambas líneas, debemos dividir el texto manualmente. La forma más fácil de hacer esto en Arduino es declarar la variable como un objeto `String` (en lugar de `char[]`) para poder usar la función `.substring()` , la cual nos permite "cortar" el texto exactamente donde queremos.

Dado que tu texto `"[PLACEHOLDER] - CC BY-SA 4.0"` tiene 28 caracteres, encaja perfectamente en los 32 espacios (16x2) de la pantalla.

Aquí tienes el código de cómo estructurarlo dentro del `setup()` :

C++



```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Cambiamos 'char[]' por 'String' para facilitar el recorte
```

```
String textoInicialA = "[PLACEHOLDER] - CC BY-SA 4.0";

void setup() {
  lcd.begin(16, 2);

  // --- PRIMERA LÍNEA (Primeros 16 caracteres) ---
  lcd.setCursor(0, 0);
  // Corta desde la posición 0 hasta antes de la 16: "[PLACEHOLDER] - "
  lcd.print(textoInicialA.substring(0, 16));

  // --- SEGUNDA LÍNEA (El resto del texto) ---
  lcd.setCursor(0, 1);
  // Corta desde la posición 16 hasta el final: "CC BY-SA 4.0"
  lcd.print(textoInicialA.substring(16));

  // Dejamos el texto visible por 4 segundos
  delay(4000);

  // Limpiamos para dar paso al siguiente mensaje o al loop principal
  lcd.clear();
}

void loop() {
  // Aquí va el código continuo
}
```

¿Por qué esto es mejor?

Control exacto: Sabes exactamente qué va en cada línea. En este caso, el primer corte termina justo en el espacio después del guion ([PLACEHOLDER] -), por lo que la segunda línea empieza limpiamente con la letra "C" de CC BY-SA 4.0 .

Escalabilidad: Puedes usar la misma lógica con textoInicialB . Como "Cuando nuestras dos almas se eleven" tiene 35 caracteres, puedes poner los primeros 16 arriba, los siguientes 16 abajo, y luego hacer una segunda pantalla para mostrar las 3 letras faltantes (o abreviar el texto).

¿Quieres que el texto de la segunda línea quede centrado?

↔ Sí

Revisa posibles errores, es importante que se siga trabajando con la variable char

```
#include <LiquidCrystal.h>
```



El error principal por el que tu código no compilará es el uso de `.substring()`. Esta es una función exclusiva de los objetos `String` en Arduino (C++). **Las variables `char` (arreglos de caracteres) no tienen la función `.substring()`**, por lo que el compilador lanzará un error indicando que esa función no existe para ese tipo de dato.

Para trabajar estrictamente con `char[]` y dividir los textos, usamos la lógica base de C++:

1. Para la primera línea, usamos un bucle `for` que imprima exactamente los primeros 16 caracteres (o hasta que se acabe la palabra).
2. Para la segunda línea, usamos aritmética de punteros (`texto + 16`), lo que le indica a Arduino que empiece a leer el arreglo directamente desde la posición 16 en adelante.

He corregido tu código, eliminado el bloque repetido del final y limpiado la lógica.

C++



```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Usar 'const' optimiza la memoria guardando esto en la memoria Flash
const char textoInicialA[] = "[PLACEHOLDER] - CC BY-SA 4.0";

// Nota: Corregí el texto repetido "nuestras nuestras".
// Aún así, este texto tiene 35 caracteres, por lo que superará los 32 espacios
const char textoInicialB[] = "Cuando nuestras dos almas se eleven";

const char textoInicialC[] = "Elizabeth Barret Brown";

void setup() {
  lcd.begin(16, 2);

  // --- PANTALLA 1: textoInicialA ---
  lcd.setCursor(0, 0);
  // Imprime letra por letra solo hasta llegar a 16 caracteres
```

```

for(int i = 0; i < 16 && textoInicialA[i] != '\0'; i++) {
    lcd.print(textoInicialA[i]);
}

lcd.setCursor(0, 1);
// Empieza a imprimir desde el carácter número 16 hasta el final
lcd.print(textoInicialA + 16);

delay(4000);
lcd.clear();

// --- PANTALLA 2: textoInicialB ---
lcd.setCursor(0, 0);
for(int i = 0; i < 16 && textoInicialB[i] != '\0'; i++) {
    lcd.print(textoInicialB[i]);
}

lcd.setCursor(0, 1);
lcd.print(textoInicialB + 16);

delay(4000);
lcd.clear();

// --- PANTALLA 3: textoInicialC ---
lcd.setCursor(0, 0);
for(int i = 0; i < 16 && textoInicialC[i] != '\0'; i++) {
    lcd.print(textoInicialC[i]);
}

lcd.setCursor(0, 1);
lcd.print(textoInicialC + 16);

delay(4000);
lcd.clear();
}

void loop() {
    // Aquí irá tu programa continuo
}

```

Un detalle importante sobre textoInicialB

Tu texto *"Cuando nuestras dos almas se eleven"* tiene 35 caracteres.

La fila 1 mostrará: Cuando nuestras (16 caracteres).

La fila 2 mostrará: dos almas se ele (16 caracteres).

Los caracteres restantes (ven) se escribirán en la memoria oculta de la pantalla y **no se verán**, ya que la pantalla física solo tiene capacidad para 32 letras estáticas. Tendrías que acortar la frase, o hacer que ese texto en específico se desplace como lo hicimos anteriormente.

El texto de la pantalla 2 (textoInicialB) puede recorrer a lo largo de la pantalla como carrusel, de manera que se pueda leer todo el texto. Considera que el carrusel ocurra en la pantalla (0, 1)

Para lograr este efecto de carrusel (desplazamiento horizontal) utilizando exclusivamente variables `char` , implementaremos una "ventana deslizante".

La lógica consiste en usar un bucle que lea la frase desde diferentes posiciones de inicio, imprimiendo siempre solo 16 caracteres. Para calcular matemáticamente cuántos "pasos" debe dar el bucle, usaremos la función nativa de C++ `strlen()` , que nos dice exactamente cuántas letras tiene el texto.

Aquí tienes el código completo con la Pantalla 2 convertida en un carrusel dinámico en la fila inferior:

C++



```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const char textoInicialA[] = "[PLACEHOLDER] - CC BY-SA 4.0";
const char textoInicialB[] = "Cuando nuestras dos almas se eleven";
const char textoInicialC[] = "Elizabeth Barret Brown";

void setup() {
  lcd.begin(16, 2);

  // --- PANTALLA 1: textoInicialA ---
  lcd.setCursor(0, 0);
  for(int i = 0; i < 16 && textoInicialA[i] != '\0'; i++) {
    lcd.print(textoInicialA[i]);
  }
}
```

```

    lcd.setCursor(0, 1);
    lcd.print(textoInicialA + 16);

    delay(4000);
    lcd.clear();

// --- PANTALLA 2: Carrusel de textoInicialB en la fila inferior (0, 1) --
int largoB = strlen(textoInicialB); // Calculamos el largo (35 letras)

// Calculamos cuántos pasos debe avanzar para mostrarlo todo.
// Si el texto es más corto de 16, no se mueve (0 pasos).
int pasosTotales = (largoB > 16) ? (largoB - 16 + 3) : 0; // +3 para dejar

for(int pos = 0; pos <= pasosTotales; pos++) {
    lcd.setCursor(0, 1);

    // Imprimimos la "ventana" de 16 caracteres
    for(int i = 0; i < 16; i++) {
        if (pos + i < largoB) {
            lcd.print(textoInicialB[pos + i]);
        } else {
            lcd.print(' '); // Rellena con espacios en blanco cuando se acaba el
        }
    }

    // Si estamos en el primer cuadro (pos = 0), hacemos una pausa más larga
    // para que el usuario pueda empezar a leer antes de que se mueva.
    if (pos == 0) {
        delay(2000);
    } else {
        delay(350); // Velocidad del carrusel (350ms por letra)
    }
}

lcd.clear();

// --- PANTALLA 3: textoInicialC ---
lcd.setCursor(0, 0);
for(int i = 0; i < 16 && textoInicialC[i] != '\0'; i++) {
    lcd.print(textoInicialC[i]);
}
lcd.setCursor(0, 1);
lcd.print(textoInicialC + 16);

delay(4000);
lcd.clear();
}

```

```
void loop() {
  // Aquí irá tu programa principal que se repetirá infinitamente
}
```

¿Cómo funciona la Pantalla 2 ahora?

1. **Pausa inicial:** Al entrar a la Pantalla 2, imprimirá estáticamente `Cuando nuestras` en la fila inferior y esperará 2 segundos (`delay(2000)`). Esto es crucial en diseño de interfaces porque si el texto empieza a correr inmediatamente, el ojo del usuario pierde las primeras palabras.
2. **Ventana deslizante:** La variable `pos` indica desde qué letra del `char` empezamos a leer. En la primera vuelta lee desde la letra 0 a la 15. En la segunda vuelta lee desde la 1 a la 16. En la tercera desde la 2 a la 17, etc.
3. **El final suave (+ 3):** Le añadí un `+ 3` a los pasos totales para que, al terminar la palabra "eleven", el texto avance unos tres espacios más y quede `ven` , dándole un cierre visual más limpio antes de limpiar la pantalla con `lcd.clear()` .

¿Quieres agregar un título fijo sobre el carrusel?

↔ Sí

→ Desde [Texto desplazable en OLED con Arduino](#)